

Il filesystem

Lezione 2

Sistemi Operativi - II Modulo
(Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica

(Materiale didattico per cortesia del Prof. Emiliano Casalicchio)



SAPIENZA
UNIVERSITÀ DI ROMA



Agenda

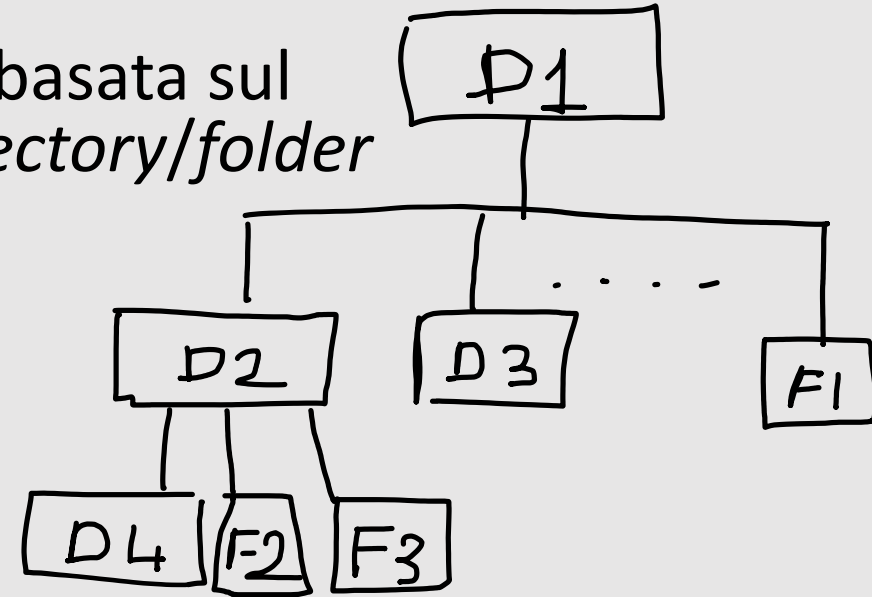
- Comandi per la gestione del file system, ovvero di file e directory

Ottimo [video di introduzione a Unix](#), dai suoi autori!



Il filesystem

- Organizzazione di un area di memoria di massa basata sul concetto di *file* (raccoglitore, schedario) e di *directory/folder* (cartella, fascicolo)
- Directory contiene file e directory
 - Struttura gerarchica ad albero
 - Solo le directory possono avere figli
 - I file sono foglie
- File *regolari* contengono sequenze di bit dell'area di memoria sulla quale c'è il filesystem
- Esistono file *non regolari*, ad es. per l'accesso di basso livello (*raw access*) a periferiche o dispositivi vari



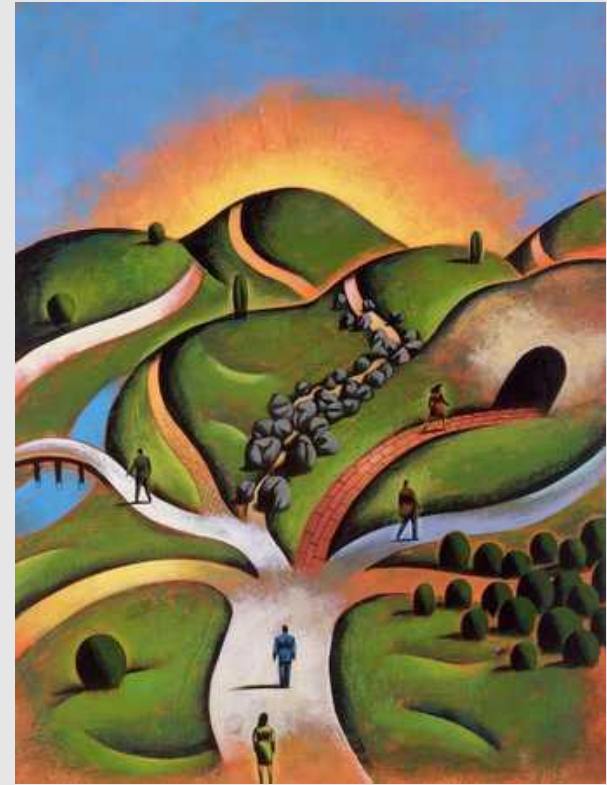
Il filesystem (cont.)

- Linux e Unix hanno un solo filesystem principale che ha come directory radice / (*root*)
 - Provate il comando `cd /`
- Tutto (file e directory) è contenuto direttamente o indirettamente in /
- In una stessa directory è vietato creare
 - 2 file con lo stesso nome
 - 2 directory con lo stesso nome
 - Un file ed una directory con lo stesso nome
- I nomi dei file e delle directory sono *case sensitive*
 - `Amore.txt` $\neq$ `amore.txt`



Cammini

- Ogni file o directory è raggiungibile dalla radice / mediante un **cammino (path) assoluto**
 - Sequenza di directory separate da /
 - `/home/utente1/dir1/dir11/dir112/file.pdf`
- Eccezione di cammino assoluto (es: per l'utente *userX*)
 - `~/dir1/dir11/dir112/file.pdf`
 - `~` equivale a `/home/userX`



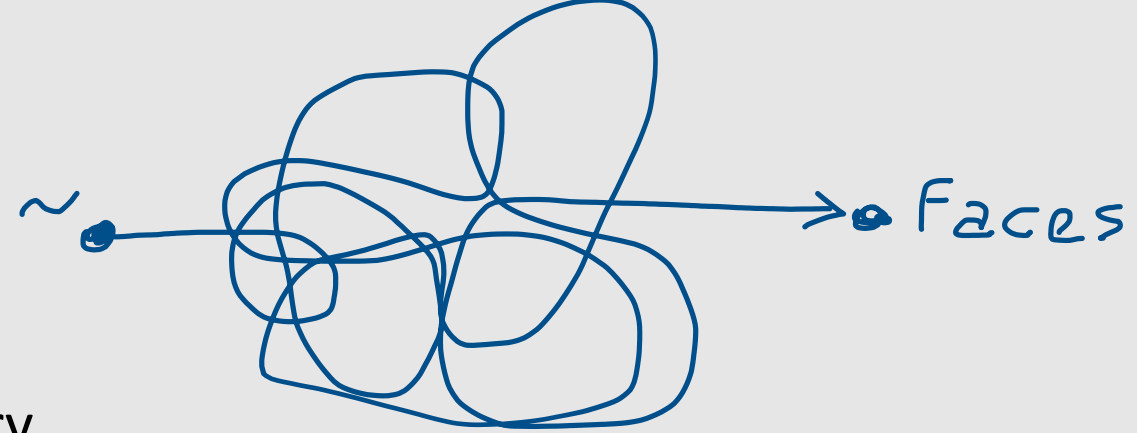
La *current working directory* (cwd)

- Per conoscere la directory in cui attualmente state lavorando, si usa `pwd`
- Per cambiare cwd usare il comando `cd [path]`
 - `[Path]` può essere assoluto o relativo
 - `cd` senza path ritorna alla home
- Nel path si possono usare `..` (directory «genitore») e `.` directory attuale

```
studente@debian9:~$ cd Immagini/faces/  
studente@debian9:~/Immagini/faces$ pwd  
/home/studente/Immagini/faces  
studente@debian9:~/Immagini/faces$ cd ..  
studente@debian9:~/Immagini$ cd .  
studente@debian9:~/Immagini$ cd  
studente@debian9:~$ cd /home/studente/Immagini/fac  
studente@debian9:~/Immagini/faces$ pwd  
/home/studente/Immagini/faces
```

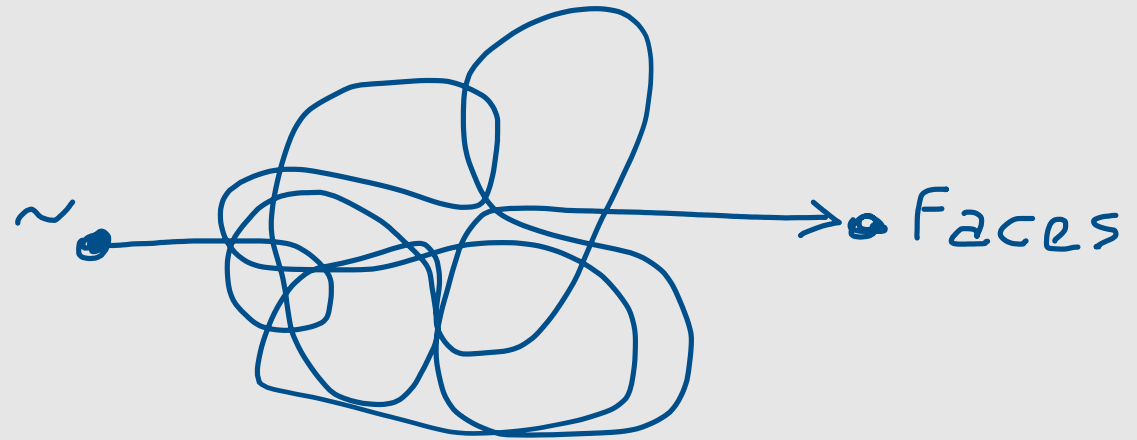


Cose strane!!!



- Usando il comando `mkdir` create le directory
 - `~/Immagini`
 - `~/Immagini/faces`
- Consultate il man per la sinossi di `mkdir`
 - Se le directory già esistono o se la directory `Immagini` già esiste non c'è bisogno di crearla
- Partendo da `~` quale sarà la `cwd` dopo il comando seguente?
 - `cd Immagini/../../Immagini/faces/`
- In quale `cwd` deve essere dato il comando seguente per far sì che la `cwd` risultante sia `~/Immagini/faces`?
 - `cd ../../../../Immagini/./faces/`

Cose strane!!! (soluzioni)



```
/home/student/Immagini/faces
studente@debian9:~/Immagini/faces$ cd
studente@debian9:~$ cd Immagini/../../Immagini/faces/
studente@debian9:~/Immagini/faces$ cd ../../../../Immagini/./faces/
studente@debian9:~/Immagini/faces$ █
```

Quale è la differenza tra cammino (path) assoluto e relativo?

- Path assoluto è valido *qualunque sia* la cwd
- Path relativo può non essere valido quando si cambia cwd

```
studente@debian9:~/Immagini$ cd /home/studente/Immagini/  
studente@debian9:~/Immagini$ cd  
studente@debian9:~$ cd Immagini/  
studente@debian9:~/Immagini$ cd faces/  
studente@debian9:~/Immagini/faces$ cd Immagini/  
studente@debian9:~/Immagini/faces/Immagini$ pwd  
/home/studente/Immagini/faces/Immagini  
studente@debian9:~/Immagini/faces/Immagini$
```

Contenuto di una directory

- Comando `ls`
 - restituisce la lista dei file contenuti in una directory
- Date un'occhiata al man per avere un'idea delle opzioni disponibili e delle modalità d'uso
- `ls`
 - Mostra contenuto `cwd`
- `ls nomedir`
 - Mostra contenuto directory `nomedir`

```
studente@debian9: ~  
File Modifica Schede Aiuto  
LS(1) User Commands  
NAME  
    ls - list directory contents  
SYNOPSIS  
    ls [OPTION]... [FILE]...  
DESCRIPTION  
    List information about the FILES (the  
    Sort entries alphabetically if none of -  
    fied.  
  
Mandatory arguments to long options  
too.
```



Contenuto di una directory (cont.)

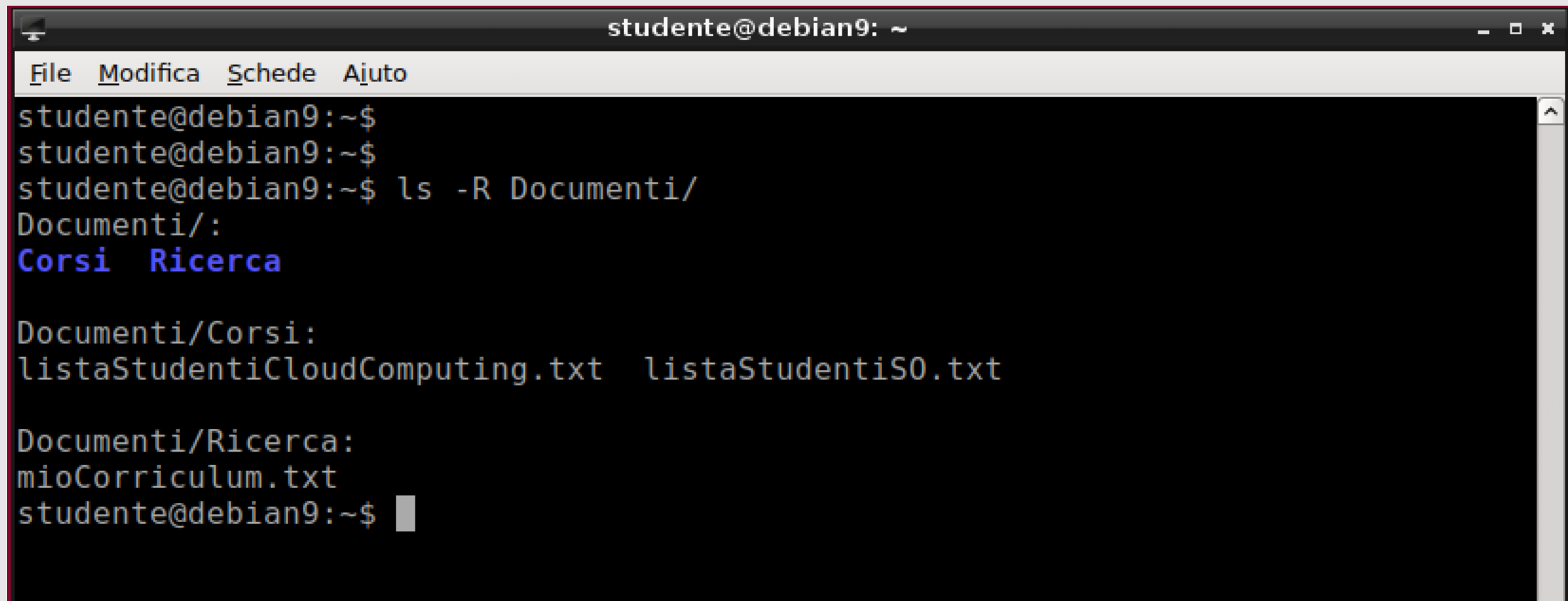
- In una directory ci sono file «nascosti»
 - Tipicamente file di configurazione o file usati a supporto di comandi e applicazioni
- ad es.
 - `.bash_history` contiene la lista dei comandi eseguiti. Provate `less .bash_history`
 - `.bashrc` contiene la configurazione della bash
- `ls [ -a | --all ]` visualizza i file nascosti

```
stude
File  Modifica  Schede  Aiuto
studente@debian9:~$ man ls
studente@debian9:~$ ls -a
.          glassfish-4.
..         .gnupg
apache-tomcat-8.0.27 .idlerc
.bash_history Immagini
.bash_logout .ipython
.bashrc      .java
.cache       .jupyter
canopy       .local
.canopy      .matlab
Canopy       Modelli
.canopy_runtimes.json .mozilla
.config      Musica
.dmrc        .nano
Documenti   .nbi
enthought   .netbeans
.enthought  netbeans-8.2
.gconf      .oracle_jre_
.gimp-2.8   .profile
```



Contenuto di una directory (cont.)

- Se vogliamo ricorsivamente visualizzare il contenuto delle sotto directory abbiamo l'opzione `[ -R | --recursive ]`
- ad esempio

A terminal window titled 'studente@debian9: ~' with a menu bar containing 'File', 'Modifica', 'Schede', and 'Ajuto'. The terminal shows a series of commands and their outputs. The user enters 'ls -R Documenti/' to recursively list the contents of the 'Documenti' directory. The output shows the directory structure and files, with subdirectories 'Corsi' and 'Ricerca' listed in blue. The files 'listaStudentiCloudComputing.txt' and 'listaStudentiS0.txt' are listed under 'Corsi', and 'mioCorriculum.txt' is listed under 'Ricerca'. The prompt returns to the user's home directory.

```
studente@debian9:~$  
studente@debian9:~$  
studente@debian9:~$ ls -R Documenti/  
Documenti/:  
Corsi  Ricerca  
  
Documenti/Corsi:  
listaStudentiCloudComputing.txt  listaStudentiS0.txt  
  
Documenti/Ricerca:  
mioCorriculum.txt  
studente@debian9:~$
```

Contenuto di una directory (cont.)

- Si può visualizzare l'albero delle directory con il comando
 - `tree [-a] [-L maxdepth] [-d] [-x] [nomedir]`
 - Se non è installato, installatelo eseguendo `sudo apt install tree`

OPTIONS

Tree understands the following command line switches:

LISTING OPTIONS

- a All files are printed. By default tree does not print hidden files (those beginning with a dot '.'). In no event does tree print the file system constructs '.' (current directory) and '..' (previous directory).
- d List directories only.
- l Follows symbolic links if they point to directories, as if they were directories. Symbolic links that will result in recursion are avoided when detected.
- f Prints the full path prefix for each file.
- x Stay on the current file-system only. Ala **find** -xdev.
- L level
Max display depth of the directory tree.
- R Recursively cross down the tree each level directories (see -L

File Modifica Schede Ajuto

studente@debian9:~\$ 3

bash: 3: comando non trovato

studente@debian9:~\$ tree -L 3

```
.
├── apache-tomcat-8.0.27
│   └── bin
│       ├── bootstrap.jar
│       ├── catalina.bat
│       ├── catalina.sh
│       ├── catalina-tasks.xml
│       ├── commons-daemon.jar
│       ├── commons-daemon-native.tar.gz
│       ├── configtest.bat
│       ├── configtest.sh
│       ├── daemon.sh
│       ├── digest.bat
│       ├── digest.sh
│       ├── service.bat
│       ├── setclasspath.bat
│       ├── setclasspath.sh
│       ├── shutdown.bat
│       ├── shutdown.sh
│       ├── startup.bat
│       └── startup.sh
```

zuliani@di.

File Modifica Schede Ajuto

146 directories, 282 files

studente@debian9:~\$ tree -d -L 3

```
.
├── apache-tomcat-8.0.27
│   ├── bin
│   ├── conf
│   ├── lib
│   ├── logs
│   ├── temp
│   ├── webapps
│   │   ├── docs
│   │   ├── examples
│   │   ├── host-manager
│   │   ├── manager
│   │   └── ROOT
│   └── work
├── canopy
│   ├── licenses
│   └── scripts
└── Canopy
    ├── appdata
    │   └── canopy-2.1.9.3717.rh6-x86_64
    └── bin
```

Creazione di directory e file

- `mkdir [-p] nomedir`
 - crea directory `nomedir` vuota
 - se volete creare un intero path di directory? Ad esempio `./dir11/dir12/dir13`
 - provate: `mkdir dir11/dir12/dir13`
 - Non funziona ... provate ora con l'opzione `-p` ?
- `touch nomefile`
 - crea file `nomefile` vuoto
 - Per modificare il contenuto di un file `gedit`, `nano`, `pico`, `vi`
 - Provate `gedit nomefile` & oppure `nano nomefile` &
 - Se `gedit` non è installato ... sapete cosa fare ... (vedi installazione del comando `tree`)

Esercizio 1

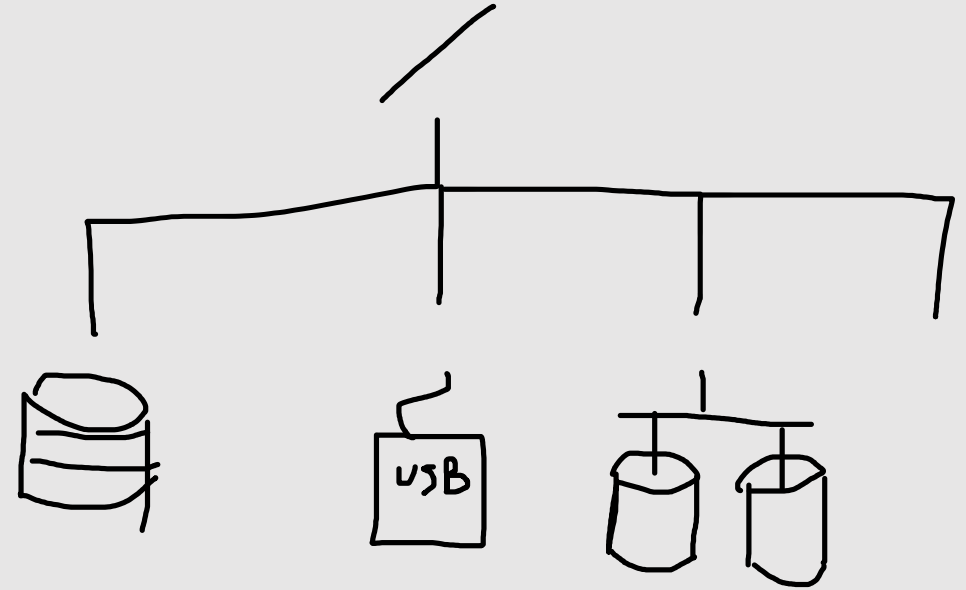
- Creare l'intero albero di directory
`/home/utente1/dir1/dir3/dir7/`
(dove `utente1` siete voi)
- Posizionarsi dentro `dir7` e poi in `dir1` sia usando sia non usando la directory parent `..`; per controllare il risultato usare il comando `ls` o `pwd`; controllare come cambia il path riportato nel prompt
- Creare un file vuoto in `dir7` e poi aprirlo e modificarlo con `gedit` (o `nano`)

Fine della I parte

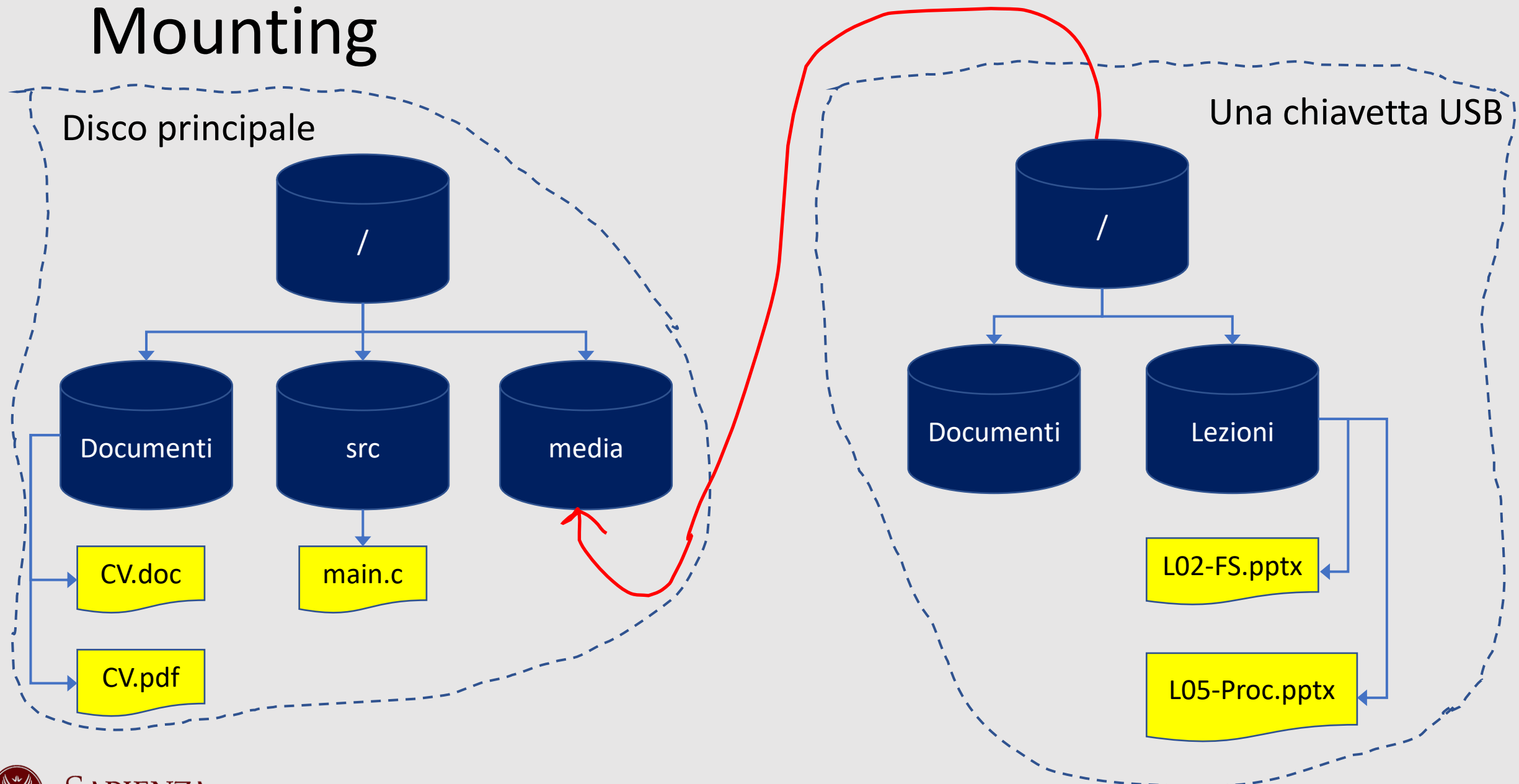


Il filesystem root (/) contiene elementi eterogenei

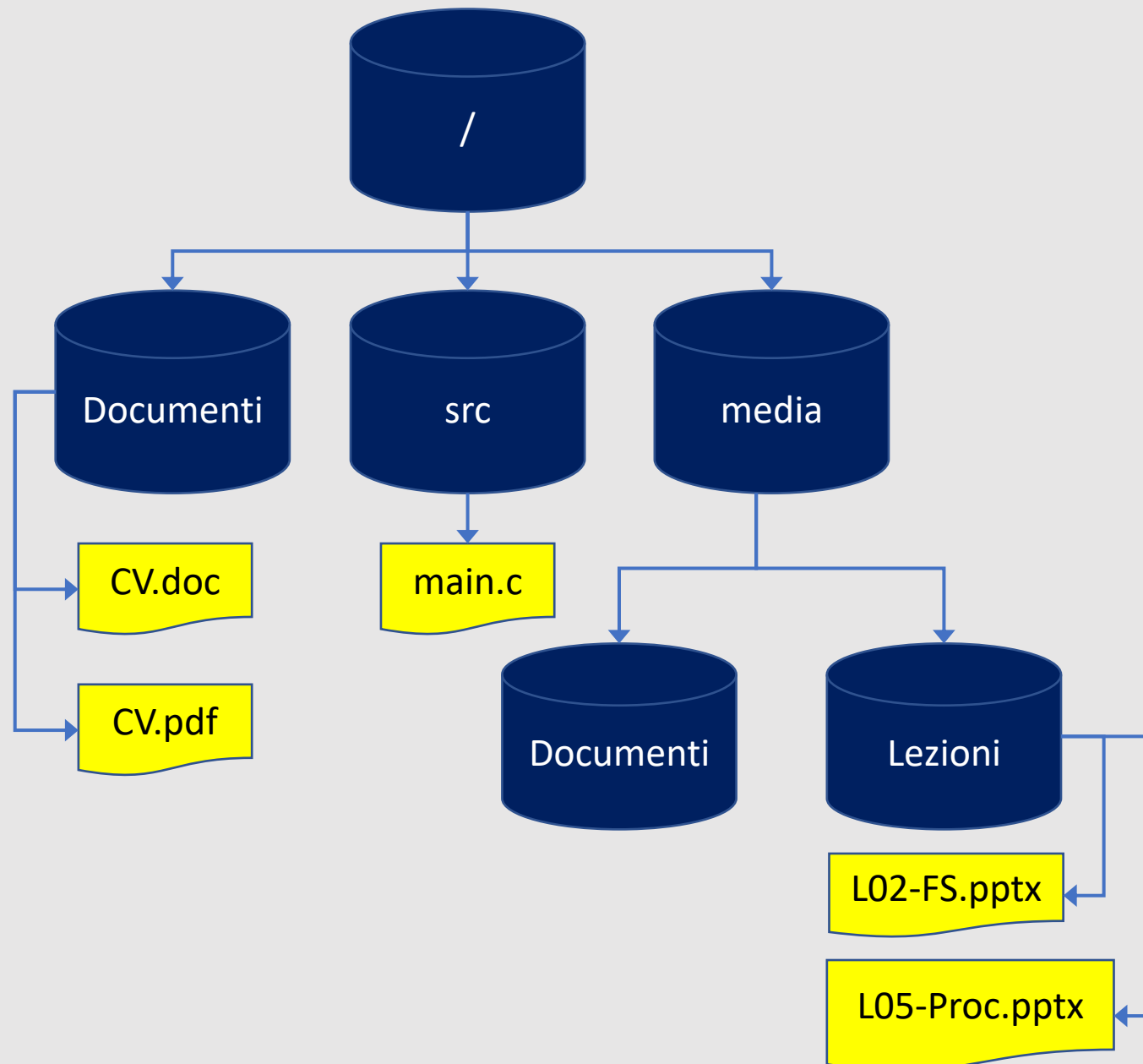
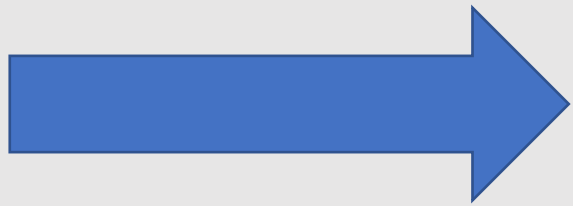
- Disco interno solido o magnetico
 - Filesystem su disco esterno (e.g., usb)
 - Filesystem di rete
 - Filesystem virtuali (usati dal kernel per gestire risorse)
 - Filesystem in memoria principale
-
- Tutto questo è possibile grazie al meccanismo del *mounting*



Mounting



Mounting



Mounting

- Una qualsiasi directory D dell'albero gerarchico può diventare il punto di mount per un altro (nuovo) filesystem F se e solo se la directory root di F diventa accessibile da D
 - Se D è vuota, dopo il mount conterrà F
 - Se D non è vuota, dopo il mount conterrà F ma ciò non significa che i dati che vi erano dentro sono persi... saranno di nuovo accessibili dopo l'unmount di F
- Una qualsiasi directory dell'albero gerarchico può diventare il punto di mount per un altro (nuovo) filesystem

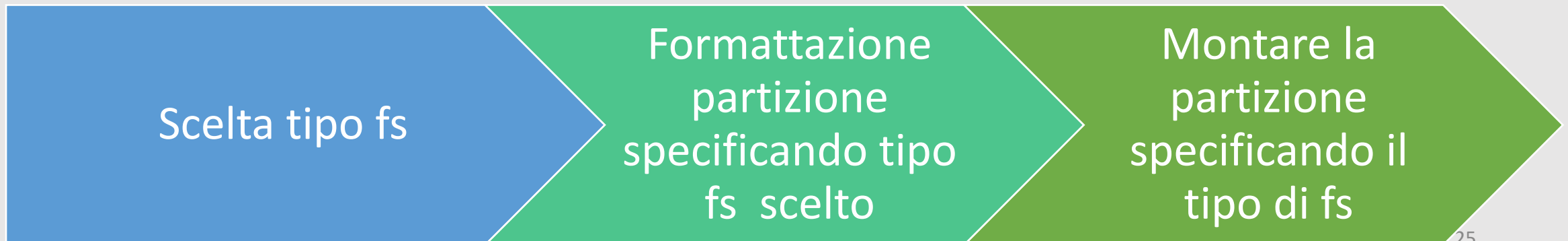
Partizioni e mounting

- Un singolo disco può essere suddiviso in due o più partizioni
- Una partizione **A** può contenere il sistema operativo e la partizione **B** i dati degli utenti (home directory degli utenti)
 - La partizione **A** verrà montata su /
 - La partizione **B** verrà montata su /home
- Quali sono i vantaggi? (*fermatevi e riflettete*)
 - Se devo reinstallare il sistema operativo in **A** non tocco i dati e rimonto **B** in /home del nuovo filesystem
 - Se voglio installare una diversa versione di Linux in un'altra partizione **C** o altro disco **D** posso sempre montare **B** su /home del nuovo filesystem senza dover ricopiare i dati

Tipi di filesystem

Nome	Journal	Partiz (TB)	File (TB)	Nome file (bytes)
Ext2	No	32	2	255
Ext3	Sì	32	2	255
Ext4	Sì	1000	16	255
ReiserFS	Sì	16	8	4032

- Dal punto di vista dell'utente
 - Dimensione max partizione
 - Dimensione max file
 - Lunghezza max nome file
 - Se Journal filesystem o no (ricordate cosa significa? Vedi I Modulo)
- Dal punto di vista del programmatore
 - Il tipo di filesystem definisce la codifica dei dati



Tipi di filesystem (cont.)

- Ci sono file system non Linux, ad esempio Windows:
 - NTFS, MSDOS, FAT32, FAT64
- FAT (16, 32, 64bit) e NTFS possono essere montati su di un fs Linux
- `mount` è il comando per montare un fs e visualizzare i fs montati
 - consultare il man, ad esempio le sezioni con le opzioni per montare filesystem FAT e NTFS

mount

- `mount` visualizza fs correntemente montati
- `cat /etc/mtab` visualizza fs montati
- `cat /etc/fstab` visualizza fs montati all'avvio (bootstrap)

`cat` permette di concatenare più files e stamparne il contenuto su standard output (stdout). Forma più semplice:

```
cat nomefile
```

Vedi man per altri opzioni

Table 2: Le tipiche directory di primo livello di un sistema Linux

Directory	Spiegazione	Montata
/boot	Kernel e file di boot	NO
/bin	Binari (programmi eseguibili) di base	NO
/dev	Devices (periferiche) hardware e virtuali	boot
/etc	File di configurazione di sistema	NO
/proc	Dati e statistiche dei processi e parametri del kernel	boot
/sys	Informazioni e statistiche di device di sistema	boot
/media	Mountpoint per device di I/O (es: CD, DVD, USB pen)	quando necessario
/mnt	(come /media)	quando necessario
/sbin	Binari di sistema	NO
/var	File variabili (log file, code di stampa, mail ...)	NO
/tmp	File temporanei	NO
/lib	Librerie	NO

Fine della II parte



File `passwd` e `group`

- `/etc/passwd`
- `/etc/group`
- Visualizzare sullo schermo il loro contenuto
 - `/etc/passwd` contiene tutti gli utenti
 - `/etc/group` contiene tutti i gruppi
- Rappresentano una delle filosofie di Linux
 - Usare file di testo (con codifica ASCII a 8 bit) con una struttura definita e conosciuta dai programmi che devono interagire con quei file
 - Ad es. il comando `adduser` conosce la struttura di questi file



Vediamo la struttura di questi file

- Organizzati per righe
- Riga = sequenza di caratteri terminata con *line feed* LF (0x0A)
- Ogni riga contiene vari campi separati da :
- `passwd` ha la seguente struttura
 - `username:password:uid:gid:gecos:homedir:shell`
 - al posto della password potete notare una `x` (cifrata)

```
studente@debian9:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
```



Vediamo la struttura di questi file (II)

- `group` ha la seguente struttura
 - `groupname:password:groupID:lista utenti`
 - Gli utenti nella Lista utenti sono separati da ,
 - Anche qui la password non è visualizzata
 - Nell'immagine a lato i gruppi sono quelli usati dal sistema e nessun utente vi appartiene
- Solitamente se una riga inizia per # significa che contiene un commento o che il suo contenuto non deve essere interpretato dalle applicazioni che usano il file

```
studente@debian9:~$ cat /etc/group
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:
tty:x:5:
disk:x:6:
lp:x:7:
mail:x:8:
```


Sgranchiamoci le dita

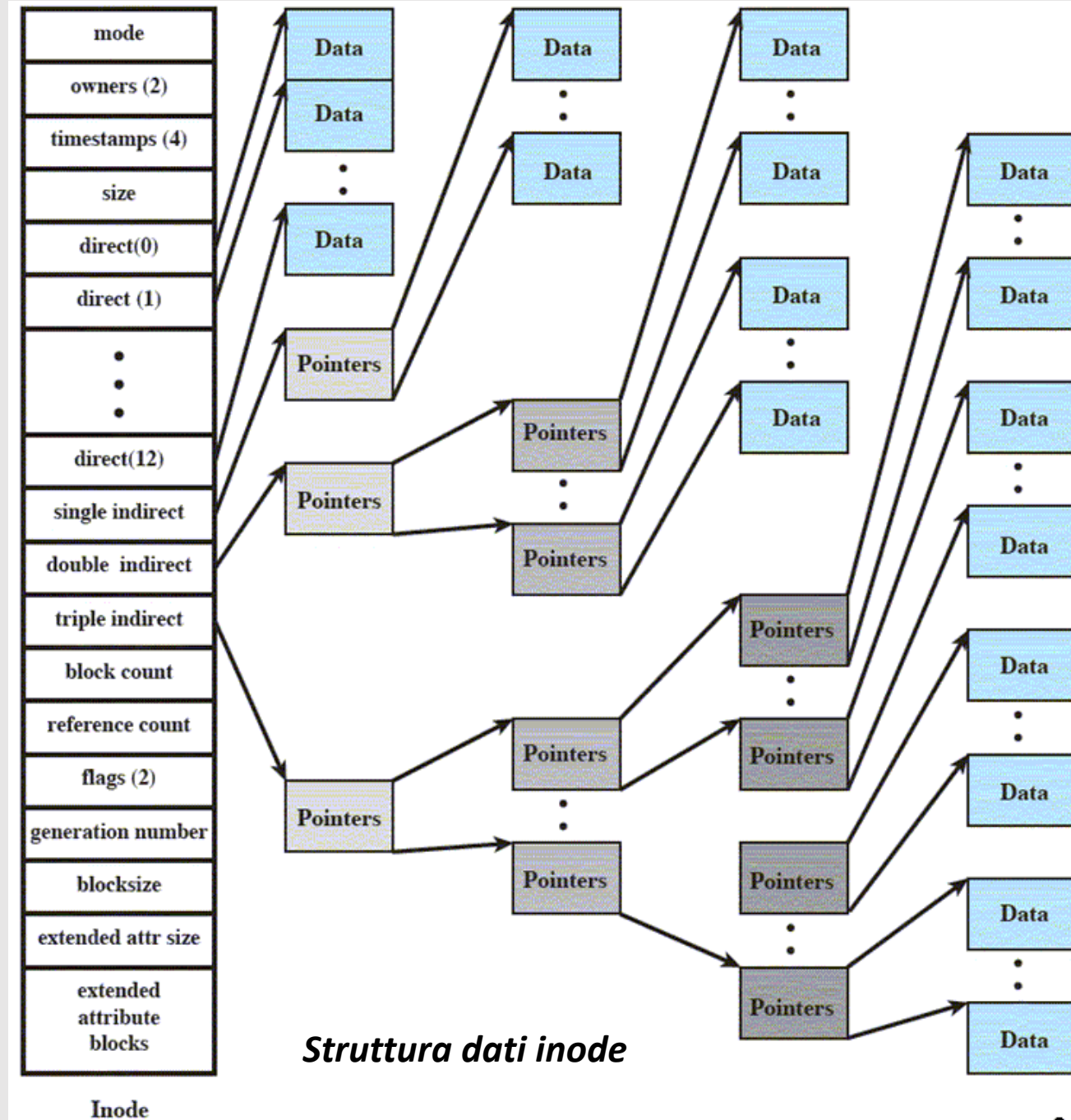
- Fate la seguente prova
- Verificare che *utente3* non sia presente in `/etc/passwd`, createlo, e poi controllare che sia presente
- Verificare che *utente3* non sia nel gruppo `sudo`, aggiungerlo al gruppo `sudo`, e poi controllare che sia presente

Fine della III parte



I file

- Ogni file nel filesystem è rappresentato da una struttura dati **inode** ed è univocamente identificato da un **inode number**
- La cancellazione di un file libera l'**inode number** che verrà riutilizzato quando necessario per un nuovo file

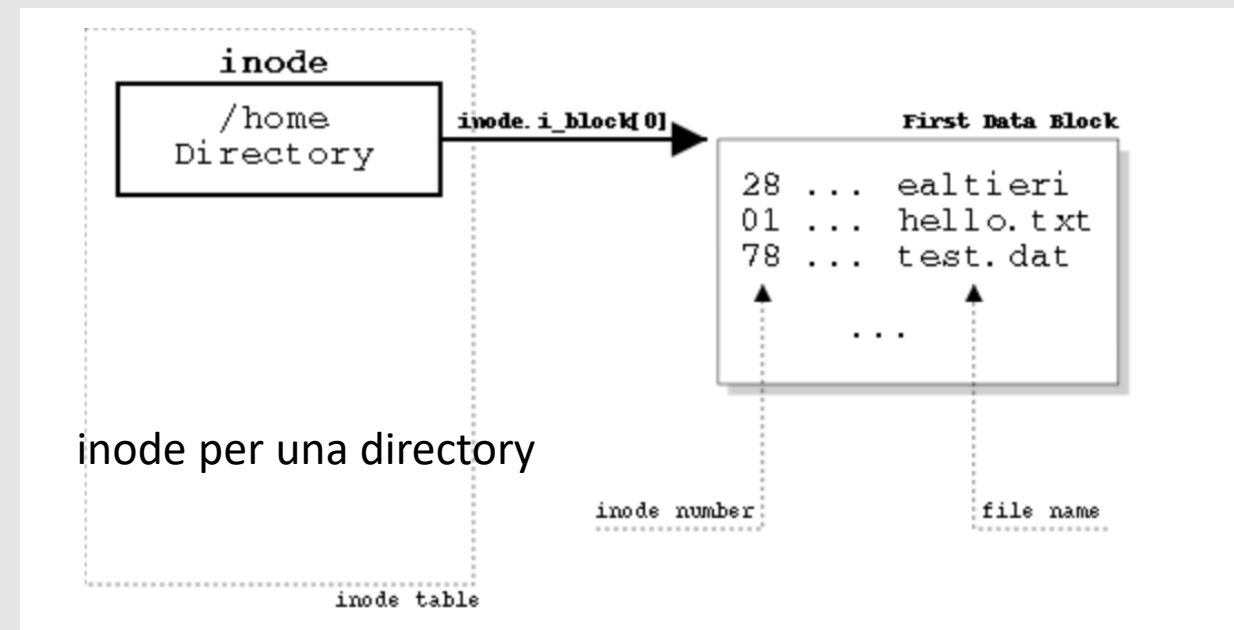
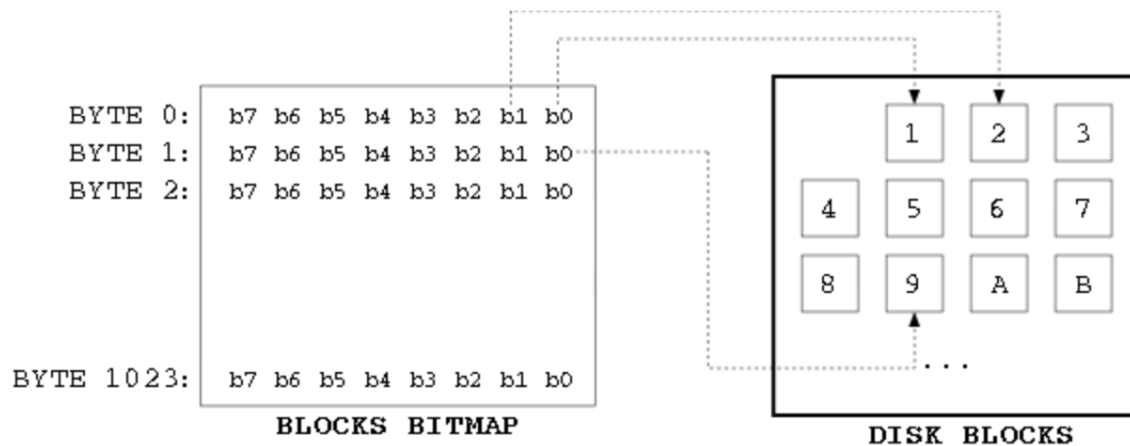
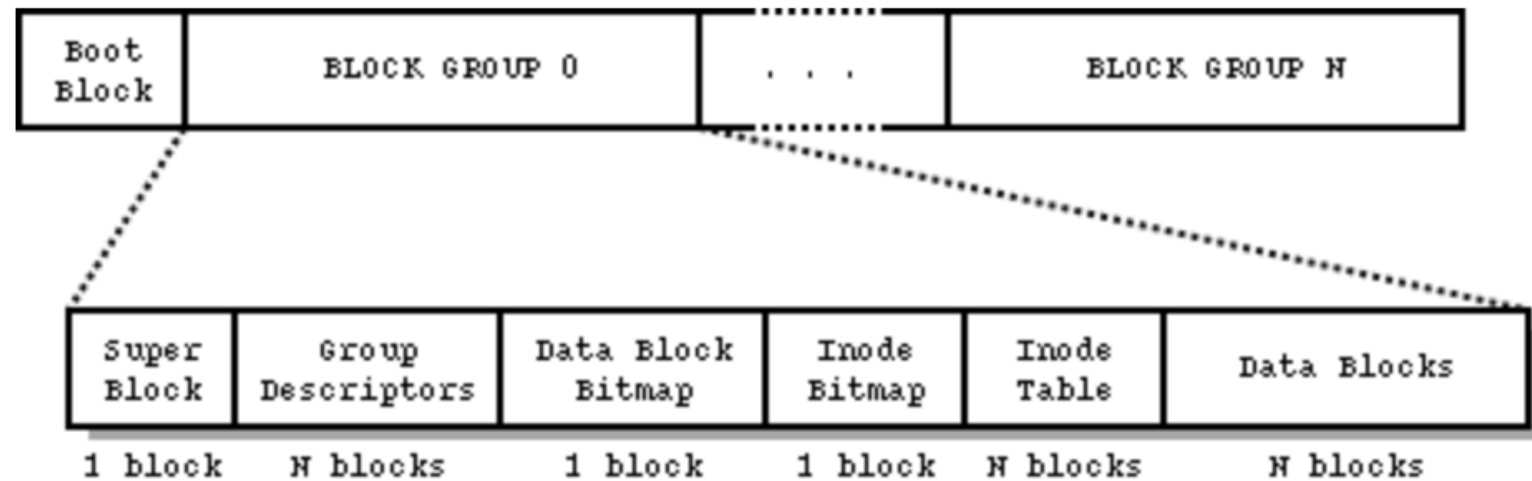


Principali attributi struttura dati inode

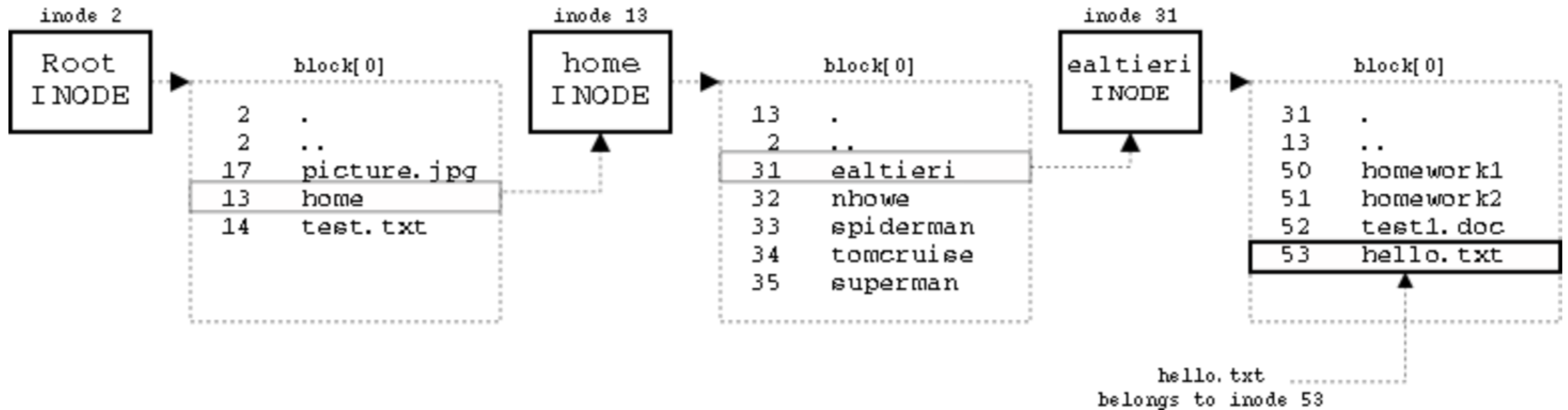
- *Type*: Tipo di file (regular, block, fifo ...)
- *User ID*: id dell'utente proprietario del file
- *Group ID*: id del gruppo a cui è associato il file
- *Mode*: Permessi (read, write, execute) di accesso per il proprietario, il gruppo e tutti gli altri
- *Size*: Dimensione in byte del file
- *Timestamps*: **ctime** (*inode changing time*: cambiamento di un *attributo*), **mtime** (*content modification time*: solo scrittura), **atime** (*content access time*: solo lettura)
- *Link count*: Numero di *hard links* (collegamenti “rigidi”)
- *Data pointers*: Puntatore alla lista dei blocchi che compongono il file; se si tratta di una directory, il contenuto su disco è costituito da una tabella con 2 colonne: nome del file/directory e suo inode number

Tabella inode

- Si trova all'inizio del disco
- Esempio per *ext2*

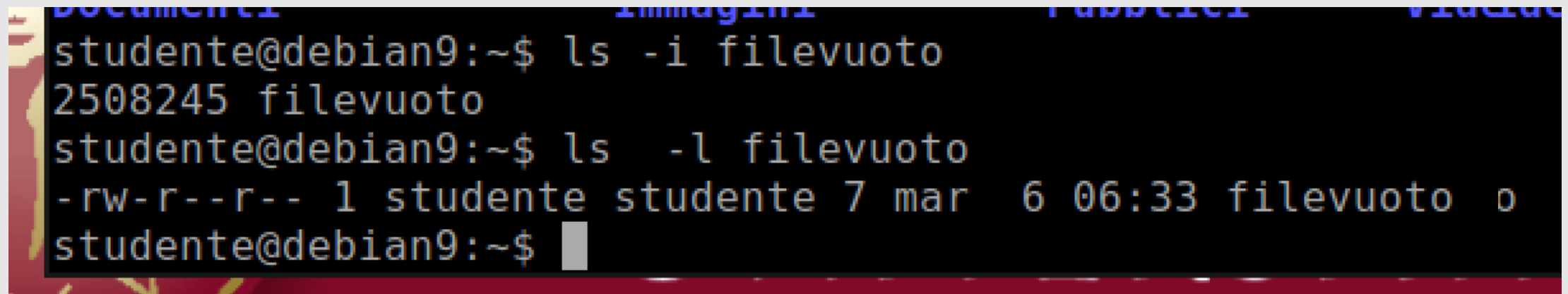


inode: come viene seguito un path



Come visualizzare le informazioni contenute nell'inode di un file?

- `ls [-a] [-c] [-u] [-R] [-l] [-i] [-n] [-S] [-h] [-1] [nomedir1] ... [nomedirn] [nomefile1] ... [nomefilen]`
- inode number: opzione `-i`
- diritti, user, group, date, size, time: opzione `-l`



```
studente@debian9:~$ ls -i filevuoto
2508245 filevuoto
studente@debian9:~$ ls -l filevuoto
-rw-r--r-- 1 studente studente 7 mar  6 06:33 filevuoto 0
studente@debian9:~$
```

ls

```
studente@debian9:~$ ls -l .
totale 64
drwxr-xr-x  9 studente studente 4096 lug 31  2018 apache-tomcat-8.0.27
drwxr-xr-x  4 studente studente 4096 lug 31  2018 canopy
drwxr-xr-x  6 studente studente 4096 lug 31  2018 Canopy
drwxr-xr-x  4 studente studente 4096 mar  6  05:48 Documenti
drwxr-xr-x  2 studente studente 4096 lug 31  2018 enthought
-rw-r--r--  1 studente studente    7 mar  6  06:33 filevuoto
```

- Dimensione
 - quanto effettivamente occupato dal file
 - per le directory dimensione del file speciale contenente lista di coppie (nomefile, inode_number) - solitamente 4K ma più file ci sono più la dimensione è grande (anche se fossero tutti file vuoti)
 - Prova a vedere le dimensione /etc o /lost+found
- Il numero dopo i diritti di accesso indica il numero di directory e file all'interno della directory (vengono contate anche . e ..) - per i file è ovviamente 1




```
studente@debian9:~$ ls -l apache-tomcat-8.0.27/
totale 1512
drwxr-xr-x 2 studente studente 4096 lug 31 2018 bin
drwxr-xr-x 2 studente studente 4096 lug 31 2018 conf

studente@debian9:~$ ls -l canopy/
totale 8
drwxr-xr-x 2 studente studente 4096 lug 31 2018 licenses
drwxr-xr-x 2 studente studente 4096 lug 31 2018 scripts
```

- **Totale:** si tratta delle dimensione della directory in blocchi su disco (normalmente, 1 blocco ha dimensione tra 1kB e 4kB)
 - riguarda solo la directory attuale, non tutto il sottoalbero
 - per ogni sottodirectory, ne considera solo la dimensione, come definita sopra (dimensione file speciale)

ls

- opzione `-n` consente di visualizzare ID utente e ID gruppo invece del nome esteso
- per vedere i timestamp sempre con l'opzione `-l`, ma in combinazione con
 - `-c` per ctime, ad es. `ls -l -c nomefile` oppure `ls -lc nomefile`
 - `-u` per atime
 - senza niente (per mtime)
- **Esercizio:** creare un file, modificarlo, poi leggerlo e verificare che i valori per atime ed mtime cambino di conseguenza

NOTA: notare che queste ed altre informazioni potrebbero non essere sono sul man. In tal caso, cercate nelle informazioni estese, reperibili con il comando `info ls`



`stat [-c format] {nomefile}`

```
studente@debian9:~$ stat filevuoto
  File: filevuoto
  Dim.: 17
  Blocchi: 8
  Blocco di IO: 4096 file regolare
Device: 801h/2048d Inode: 2508250    Coll: 1
Accesso: (0644/-rw-r--r--)  Uid: ( 1000/studente)  Gid: ( 1000/studente)
Accesso  : 2020-03-07 22:36:52.880272001 +0100
Modifica : 2020-03-07 22:36:52.880272001 +0100
Cambio   : 2020-03-07 22:36:52.880272001 +0100
Creazione: -
studente@debian9:~$ stat -c %B filevuoto
512
```

- Ad es:
 - `stat filename` restituisce varie informazioni
- `stat -c %B filename` restituisce la dimensione del blocco su disco che coincide con la dimensione di un settore di disco
 - `sudo fdisk -l /dev/sdax` dove `x=1, 2, ...`

Fine della IV parte

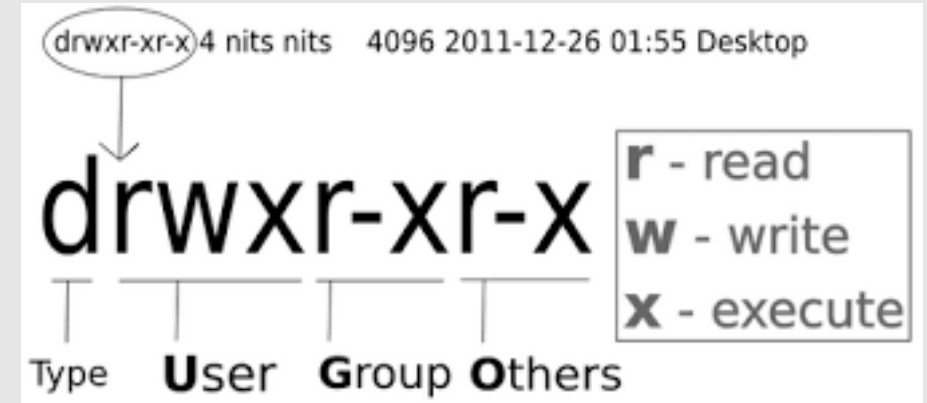


Permessi di accesso ai file: chi può fare cosa

- **Utente proprietario:** solitamente chi crea il file/directory
- **Gruppo proprietario:** gruppo primario dell'utente proprietario (specificato in `/etc/passwd`)
- Proprietario definisce i permessi di accesso: chi può leggere, scrivere, eseguire un file/directory



Permessi di accesso a file



	4	2	1	
0	-	-	-	no permissions
1	-	-	x	only execute
2	-	w	-	only write
3	-	w	x	write and execute
4	r	-	-	only read
5	r	-	x	read and execute
6	r	w	-	read and write
7	r	w	x	read, write and execute

Scrittura significa

- Sovrascrivere
- Append (scrivere alla fine)
- Cancellare (ma servono specifici diritti sulla directory)

Permessi di accesso a directory

Permesso	Ottale	Significato
- - -	0	Non si può fare niente
- - x	1	Si può impostare come cwd (ma solo se il permesso c'è per tutte le directory nel path); si può anche "attraversare", se già se ne conosce il contenuto
- w -	2	Non si può fare niente
- w x	3	Come il permesso 7, ma non si può elencare il contenuto
r - -	4	Solo elencare il contenuto (senza vedere attributi file), non può essere «attraversata»
r - x	5	Si può leggere (attributi compresi), impostare come cwd ed attraversare; non è possibile cancellare o aggiungere file/directory
r w -	6	Come il permesso 4
r w x	7	Elencare contenuto (attributi compresi), aggiungere file e directory, cancellare file contenuti in essa (anche senza avere il permesso di scrittura sul file! Correggibile con lo <i>sticky bit</i> , vedere più avanti), cancellare directory contenute in essa (ma occorrono tutti i permessi su tali directory)

Permessi speciali

- sticky bit (t)
- setuid bit (s)
- setgid bit (s)
- Possono essere applicati a file e directory

Sticky bit (t)

- Inutile sui file
- Applicato su directory corregge il comportamento di $w+x$ permettendo la cancellazione di file se si hanno permessi di scrittura su di essi

Sticky bit (t)

- Siano
 - D una directory,
 - Usr1 e Usr2 due utenti $Usr1 \neq Usr2$
 - D appartiene a Usr1
 - D non appartiene a Usr2 ne' al gruppo di Usr2
 - f un file in D
- Se Usr2 cerca di cancellare f allora,
 - Senza sticky bit su D, sarà sufficiente avere i diritti di scrittura su D (nel Gruppo "other"...) per cancellare f, anche se non si hanno i permessi di scrittura su f (sempre su "other").
 - Con sticky bit, è necessario anche il permesso di scrittura su f per cancellare f.

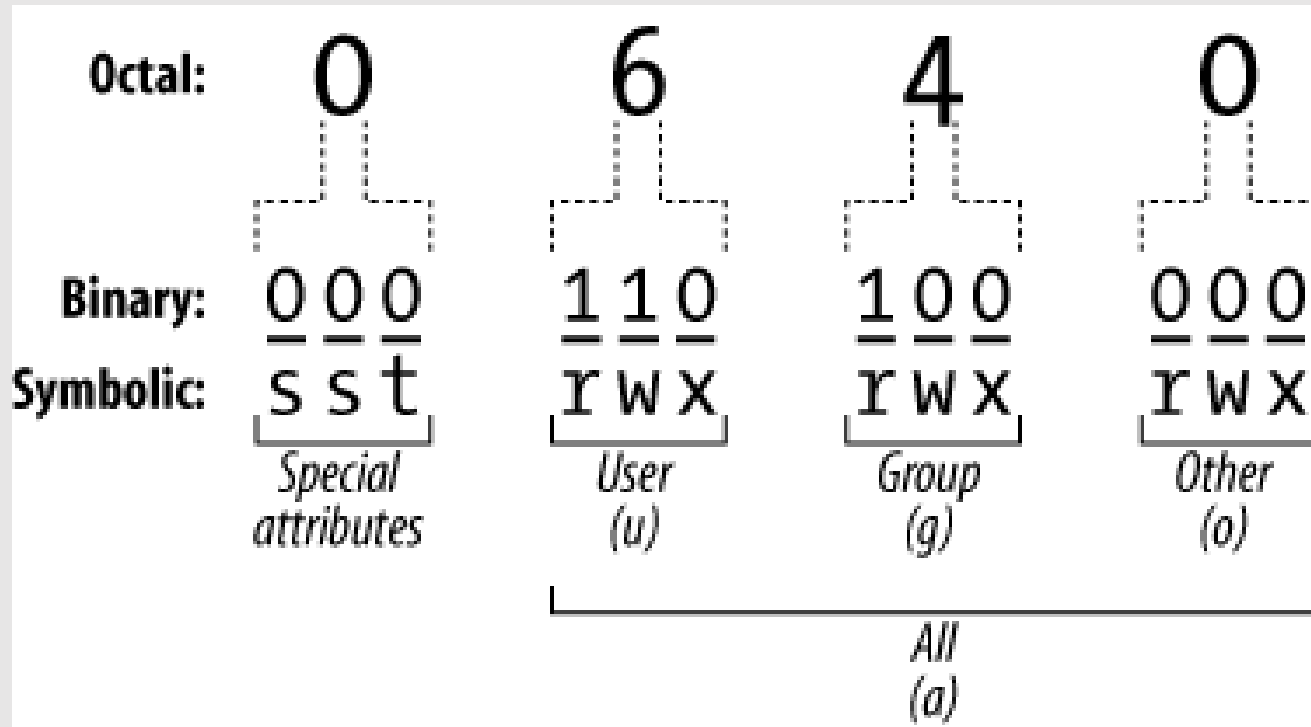
Setuid bit (s)

- Si usa solo per i file *eseguibili*
 - quando vengono eseguiti, i privilegi con cui opera il corrispondente processo non sono quelli dell'utente che esegue il file, bensì quelli dell'utente proprietario del file
 - quindi, se il proprietario è root, viene eseguito con i privilegi di root, indipendentemente da chi lo ha eseguito
- Ad esempio, il comando `passwd` ha il setuid, che permette ad un utente di modificare la propria password
 - Il proprietario del comando è root!
- setuid può essere troppo ‘potente’, quindi si cerca di limitarlo:
 - <https://blogs.oracle.com/solaris/post/when-setuid-root-no-longer-means-setuid-root-forced-privileges>

Setgid bit (s)

- Analogo di setuid ma con i gruppi
 - i privilegi sono quelli del gruppo che è proprietario del file eseguibile
- Può essere applicato anche ad una directory, e allora ogni file creato lì dentro ha il gruppo della directory, anziché quello primario di chi crea i files

Visualizzazione attributi di accesso



- Con comando `ls o stat`

```
studente@debian9:~$ ls -l filevuoto
-rw-r--r-- 1 studente studente 17 mar 7
studente@debian9:~$
```

```
studente@debian9:~$ stat filevuoto
  File: filevuoto
  Dim.: 17                Blocchi: 8
Device: 801h/2049d      Inode: 2508250
Accesso: (0644/-rw-r--r--)  Uid: ( 1000/
Accesso  : 2020-03-07 22:36:52.880272001
Modifica : 2020-03-07 22:36:52.880272001
Cambio   : 2020-03-07 22:36:52.880272001
Creazione: -
studente@debian9:~$
```

Visualizzazione permessi speciali

- Vengono visualizzati al posto del bit di esecuzione:
 - il setuid nella terna utente (user)
 - il setgid nella terna gruppo (group)
 - lo sticky nella terna altro (other)
- Provate il comando
 - `stat /tmp /usr/bin/passwd`
- Se il permesso di esecuzione c'è, allora la `s` o la `t` saranno minuscoli, altrimenti saranno Maiuscoli



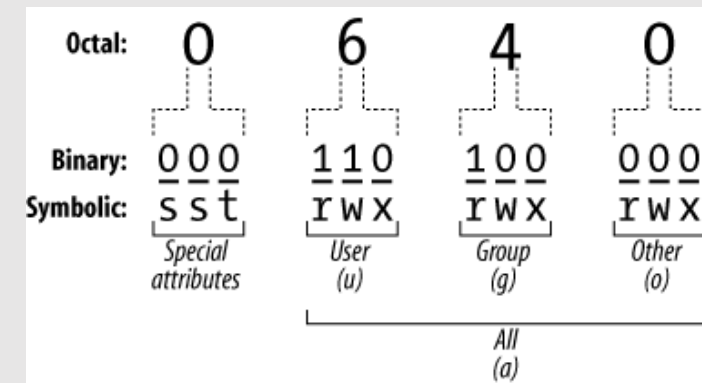
Esercizio 2

- Capire perché il comando `adduser` va usato solo da un utente amministratore, mentre il comando `groups` no



chmod mode[, mode...] filename

- Cambia la modalità (diritti) di accesso a file o directory
- Formato ottale
 - 4 numeri tra 0 e 7, come nelle Tabelle viste nelle precedenti slides;
 - il primo numero indica setuid (4), setgid (2) e sticky (1), gli altri sono per utente, gruppo ed altri



- Si possono fornire 3 numeri se si assume setuid, setgid e sticky impostati a 0

Esercizio 3

5 minuti

creare un file e, usando la modalità ottale, impostarne i permessi a

- `rw- r-- r-`

poi a

- `rws r-S -w-`

ed infine a

- `rwX r-- -wT`



chmod - modalità simbolica (lettere)

- Per impostare il mode usando dei simboli il formato è
 - `[ugoa] [+ -=] [perms . . .]`
- Dove `perms` è
 - zero
 - una o più lettere nell'insieme `{rxwXst}`
 - oppure una lettera nell'insieme `{ugo}`
- Capire dal manuale come funziona (attenzione alla `a`, che tiene conto del masking, di cui si tratterà più avanti)

Esercizi 4

10 minuti

1. Creare un file e, usando la modalità simbolica, impostarne i permessi a
 - `rws r-S -w-` e poi a
 - `rwX r-- -wT`
2. Togliere il permesso di lettura ad un file di testo, e poi provare ad aprirlo con `gedit/vi/nano`
3. Il comando `chmod` modifica il `ctime` del file, verificarlo

Precisazione su `ctime`: dovrebbe essere modificato ogni volta che cambiano gli attributi. C'è però un'eccezione: se si accede in lettura al file si modificano gli attributi, perché viene modificato il timestamp di accesso; tuttavia, il `ctime`, in questo caso, non viene modificato

```
chown [-R] proprietario {file}  
chgrp [-R] gruppo {file}
```

- Possono essere usati solo da `root` (richiedono quindi `sudo`)
- Se `file` è una directory con l'opzione `-R` si applica il comando a tutte le sotto directory
- Esercizio 5:
 - riprendendo l'esempio (Esercizio 1) dell'albero di directory `/home/utente1/dir1/dir3/dir7/`, creare un file (vuoto) `filei` dentro ciascuna directory `diri` ($i=1, 3, 7$), e cambiare i proprietari in questo modo:
 - la directory `dir3` diventa di `utente3`,
 - tutti i file dentro `dir3` diventano di `utente2`,
 - `dir7` diventa di `utente2` e `file7` diventa di `utente3`.
 - Dopodiché, provare a cambiare i permessi di `file3` e `dir7`.

Esercizio 6

10 minuti

- Creare una nuova directory e copiarci (o spostarci) dentro
 - `create_dirs_and_files.script`
 - lo script è disponibile su classroom/Lezione 03
- Eseguire con il seguente comando:
 - `bash create_dirs_and_files.script`
- Eseguire `ls -lR`
- Come si può vedere, ora ci sono 8 file, uno per ogni permesso, e 8 directory, una per ogni permesso all'interno di ciascuna directory, si ripete lo schema: 8 file e 8 directory, ciascuna di queste ultime con dentro 8 file
- Ora testare i vari permessi, come sono riportati nella tabella introdotta precedentemente. Ad esempio, provare a leggere un file solo eseguibile, o ad appendere testo ad un file solo leggibile, o ad attraversare una directory senza permesso di esecuzione...



- **Esercizio 7:** provare a fare un esempio funzionante di sticky bit. A tal proposito, creare da root (usando sudo) una directory avente tutti i diritti per tutti i gruppi, e crearci dentro un file, togliendo a quest'ultimo i permessi per il gruppo “other”; infine, provare a cancellarlo. Fare lo stesso dopo aver aggiunto lo sticky bit alla directory.
- **Esercizio 8:** provare a fare un esempio funzionante di setuid bit. A tal proposito, copiate */bin/cat* in una directory, e abilitategli il setuid bit. Create un file da root (usando sudo), e dategli il solo permesso di lettura per il solo gruppo «owner». Provare a vedere il contenuto di questo file usando il comando *cat* di sistema: fallirà. Ora provate a farlo usando *./cat...*

Domande?

